PAULT DOCUMENTATION

2026 02 19 Pro Features Design

Source: docs/plans/2026-02-19-pro-features-design.md

> **Theme:** Clean Technical

Generated: February 26, 2026

Pault Pro Features — Design Document

Date: 2026-02-19

Status: Approved for implementation planning

Author: Brainstorming session

Context

Pault is a mature local-first macOS prompt library (Phase 2.5 complete, pre-QA polish done). The goal of this design phase is to identify a coherent set of Pro features that justify a subscription price point of $8–12/month for individual users and a higher tier for teams.

Target customers:

- **Individual Pro**: Power users of AI tools (ChatGPT, Claude, Cursor daily users). They want workflow depth and speed that free users don't need.
- **Team Pro**: Enterprise/org teams standardizing AI usage across the organization.

Design philosophy: Ship one strong feature from each category — AI intelligence, in-app execution, workflow automation, and sync — so the Pro pitch is a coherent *platform*, not a checklist. Each feature is independently useful; together they differentiate Pault as the professional AI workflow tool for macOS.

Feature Overview

| Pillar | Feature | Tier |

|--------|---------|------|

| AI Assist | Improve, Suggest Variables, Auto-tag, Quality Score | Individual Pro |

| API Runner | Run prompts directly against LLM in-app | Individual Pro |

| Prompt Chains | Chain prompts end-to-end; Shortcuts integration | Individual Pro |

| Sync | iCloud sync (Apple teams) + Git-backed library (dev teams) | Team Pro |

| Pro Gating | StoreKit 2 paywall with free trial | Both tiers |

Pillar 1: AI Assist

Purpose

Turn Pault from passive storage into an active collaborator. Users can improve, analyze, and parameterize their prompts without leaving the app.

Four Operations

| Operation | Description | Output |

|----------|-------------|--------|

| **Improve** | Rewrites prompt content using a user instruction ("Make more specific", "Add CoT") | Before/after diff; accept or reject |

| **Suggest Variables** | Scans prompt text and suggests `{{placeholder}}` tokens for repeated literals | Inline annotations; click to apply |

| **Auto-tag** | Classifies prompt and suggests 1–3 tags | Dismissible tag chips |

| **Quality Score** | Rates on Clarity, Specificity, Role Definition, Output Format (1–10 each + one-line reason) | Read-only scorecard |

UI

AI Assist is a **collapsible panel** in `EditPromptView`, toggled by a `sparkles` toolbar button. The panel is ~200pt tall, sits below the `RichTextEditor`, and never appears in the popover or hotkey launcher (edit-mode only). It contains tabs for each operation.

API Key Management

- **Storage**: Keychain via `SecItemAdd` / `SecKeychainFindGenericPassword` — never `AppStorage`
- **Configuration**: Settings → AI tab (new tab in `PreferencesView`)
- Provider picker: Claude / OpenAI / Ollama (custom URL)
- API key field (masked input, stored to Keychain on save)
- Model picker per provider (e.g., `claude-opus-4-6`, `gpt-4o`)
- **Privacy**: On first use, show a sheet confirming only prompt text is sent — no metadata, tags, or IDs

New Files

- `Pault/Services/AIService.swift` — `AIService` actor: `improve(prompt:instruction:)`, `suggestVariables(for:)`, `autoTag(prompt:existingTags:)`, `qualityScore(for:)` — all `async throws`
- `Pault/Services/KeychainService.swift` — `KeychainService` struct: `save(key:value:)`, `load(key:) -> String?`, `delete(key:)`
- `Pault/Views/AIAssistPanel.swift` — the collapsible panel view
- `Pault/Views/QualityScoreView.swift` — scorecard display component

Modified Files

- `Pault/Views/EditPromptView.swift` — add `AIAssistPanel` and toolbar toggle
- `Pault/PreferencesView.swift` — add AI tab with provider/key/model config

Pillar 2: API Runner

Purpose

Execute a prompt directly against an LLM without switching apps. Fill variables, click Run, read the response in a streaming panel.

UX Flow

1. User opens a prompt in `PromptDetailView`

2. **Run** button appears alongside **Copy** (only visible when API key is configured)

3. Variables are filled in the existing `InlineVariablePreview` UI

4. Click Run → template is resolved → request is sent → response streams into **Response Panel** below the detail view

5. Response Panel shows: model name, token count estimate, **Copy Response** button, **Save as New Prompt** button

6. Response is **ephemeral** — not persisted unless user clicks Save

Streaming

Uses `URLSession` with async byte streaming (`AsyncBytes`). Response text updates a `@State var streamingResponse: String` token-by-token. A `@State var isRunning: Bool` controls a cancel button.

Error Handling

Inline error banner (not modal) with reason: invalid key, rate limit, network failure, context length exceeded. Retry button.

New Files

- `Pault/Views/ResponsePanel.swift` — streaming response display
- (Reuses `AIService` from Pillar 1)

Modified Files

- `Pault/Views/PromptDetailView.swift` — add Run button and ResponsePanel

Pillar 3: Prompt Chains

Purpose

String prompts into a pipeline. Output of step N is input to step N+1. Run the full chain with one click; expose chains to Apple Shortcuts.

Data Model

```
@Model class Chain {
var id: UUID
var name: String
var steps: [ChainStep] // ordered
var createdAt: Date
var updatedAt: Date
}

@Model class ChainStep {
var id: UUID
var sortOrder: Int
var prompt: Prompt // SwiftData relationship
var variableBindings: [String: ChainBinding] // serialized as JSON string
}

enum ChainBinding: Codable {
```

```
case literal(String) // hardcoded value
case previousOutput // pipe previous step's response
case userInput // prompt user at run time
}
```

UI — Chain Editor

- **Sidebar**: New "Chains" section below the existing filter rows (Recently Used, All Prompts, Archived, Tags)
- **Chain List**: Shows chain name + step count; click to open Chain Editor
- **Chain Editor**: Vertical stack of step cards. Each card shows: prompt title, variable binding list (each variable → binding type). Drag to reorder. + button opens a prompt picker.
- **Run Chain button**: Executes sequentially, shows step-by-step progress, displays final response in `ResponsePanel`

Shortcuts Integration (App Intents)

```
struct RunChainIntent: AppIntent {
static var title: LocalizedStringResource = "Run Prompt Chain"
@Parameter(title: "Chain Name") var chainName: String
@Parameter(title: "Initial Input") var initialInput: String?

func perform() async throws -> some ReturnsValue<String> {
let output = try await ChainRunner.shared.run(chainName: chainName, input: initialInput)
return .result(value: output)
}
}
```

This exposes "Run Prompt Chain" as a Shortcuts action. Users can build multi-app workflows entirely in Shortcuts.app.

New Files

- `Pault/Models/Chain.swift` — `Chain` and `ChainStep` SwiftData models
- `Pault/Services/ChainRunner.swift` — sequential execution engine
- `Pault/Views/ChainListView.swift` — sidebar chain list
- `Pault/Views/ChainEditorView.swift` — chain editor
- `Pault/Intents/RunChainIntent.swift` — App Intents integration
- `Pault/Intents/CopyPromptIntent.swift` — expose individual prompts to Shortcuts too

Modified Files

- `Pault/SidebarView.swift` — add Chains section
- `Pault/PaultApp.swift` — register `ModelContainer` with new Chain/ChainStep models

Pillar 4: Sync

4A: iCloud Sync

Mechanism: SwiftData's native CloudKit integration.

In `PaultApp.swift`, update `ModelConfiguration` to include a CloudKit container:

```
let config = ModelConfiguration(
isStoredInMemoryOnly: false,
```

```
cloudKitDatabase: .private("iCloud.com.pault.app")
)
```

This enables automatic bidirectional sync for all `@Model` types across devices on the same iCloud account. Conflict resolution is "last write wins" (SwiftData default).

Settings UI: Settings → Sync tab (new)

- "iCloud Sync" toggle with explainer
- Sync status: last synced timestamp, conflict count
- "Sync Now" button (manual trigger)

Team use case: Small teams share a single organizational iCloud account, or use iCloud Drive shared folder + Git (see 4B).

4B: Git-backed Library

Mechanism: A `GitSyncManager` service reads/writes prompts as `.md` files to a user-configured directory, which is itself a local Git repo.

On-disk format (`~/<repo>/prompts/<id>.md`):

```
---
id: "3F4E5A..."
title: "Summarize article"
tags: [research, writing]
favorite: true
variables:
- name: article_text
default: ""
---

Summarize the following article in 3 bullet points:
{{article_text}}
```

Operations:

- **Pull**: Read `.md` files → parse YAML frontmatter → upsert into SwiftData
- **Push**: Write SwiftData prompts → `.md` files → `git add -A && git commit -m "Pault sync [timestamp]"` → optionally `git push origin main`
- **Remote**: User configures remote URL in Settings; pull/push use system git credentials (SSH or HTTPS)

Settings UI (same Sync tab as 4A):

- "Git Library" section with repo path picker, remote URL text field
- Manual Pull / Push buttons + last sync timestamp
- Sync status badge in sidebar footer

New Files:

- `Pault/Services/GitSyncManager.swift` — pull/push operations via `Process` shell commands
- `Pault/Services/PromptMarkdownSerializer.swift` — YAML+Markdown encode/decode

Modified Files:

- `Pault/PaultApp.swift` — optional CloudKit config path
- `Pault/PreferencesView.swift` — Sync tab

Pillar 5: Pro Gating (StoreKit 2)

Products

| Product ID | Tier | Price |

|---|---|---|

| `com.pault.pro.monthly` | Individual Pro | $9.99/mo |

| `com.pault.pro.annual` | Individual Pro | $79.99/yr |

| `com.pault.team.monthly` | Team Pro | $19.99/mo per seat |

Architecture

```
@Observable class ProStatusManager {
static let shared = ProStatusManager()
private(set) var isProUnlocked: Bool = false
private(set) var isTeamUnlocked: Bool = false

init() {
Task { await listenForTransactions() }
}

private func listenForTransactions() async {
for await result in Transaction.updates {
// update isProUnlocked / isTeamUnlocked
}
}
}
```

Paywall UI

When a free user taps a Pro feature: a `PaywallView` sheet slides up showing:

- Feature name and description ("You discovered: API Runner")
- Feature preview screenshot
- Monthly / annual price toggle
- Primary CTA: "Start 7-day free trial"
- Restore Purchases link

Pro features are annotated with a `@ProRequired` enforcement point — a shared `proGate()` function that checks `ProStatusManager.shared.isProUnlocked` and presents the paywall if false.

New Files

- `Pault/Services/ProStatusManager.swift` — StoreKit 2 entitlements observer
- `Pault/Views/PaywallView.swift` — paywall sheet
- `Pault/Views/ProBadge.swift` — small "PRO" badge component for gated UI elements

Implementation Order

These pillars have dependencies. Recommended build order:

1. **Pro Gating infrastructure** — `ProStatusManager`, `PaywallView`, `ProBadge` — needed to gate everything else

2. **AI Service + Keychain** — foundation for Pillars 1 and 2
3. **AI Assist panel** (Pillar 1) — uses AIService; self-contained in EditPromptView
4. **API Runner** (Pillar 2) — uses AIService; extends PromptDetailView
5. **Prompt Chains** (Pillar 3) — new data models + execution engine + Shortcuts intents
6. **Sync — iCloud** (Pillar 4A) — modify ModelContainer config; lower risk
7. **Sync — Git** (Pillar 4B) — new service + serializer; higher complexity

---

Technical Risks

| Risk | Mitigation |

|---|---|

| CloudKit schema migrations are destructive | Enable iCloud sync only after establishing a stable data model; document migration runbook |

| StoreKit sandbox testing requires real Apple ID | Use `StoreKit Configuration File` for local testing without real purchases |

| Git shell commands via `Process` may fail silently | Wrap all `Process` calls in `AsyncStream` capturing stdout/stderr; surface errors in UI |

| App Intents require macOS 13+ | Gate `RunChainIntent` availability with `@available(macOS 13, *)` check |

| AI streaming can leave dangling `URLSession` tasks | Maintain a `currentTask: Task<Void, Never>?` reference; cancel on view disappear |

Verification

- **AI Assist**: EditPromptView → sparkles button → Improve tab → paste instruction → confirm diff renders; reject → content unchanged
- **API Runner**: PromptDetailView → fill variables → Run → confirm streaming response appears; cancel mid-stream → confirm request is aborted
- **Prompt Chains**: Create 2-prompt chain with previous output binding → Run Chain → Step 2 receives Step 1's output correctly; trigger "Run Prompt Chain" from Shortcuts.app → confirm output returned
- **iCloud Sync**: Enable on Device A → create prompt → verify appears on Device B within ~30s; offline create on Device A → reconnect → verify sync
- **Git Sync**: Push → verify `.md` files appear in repo with correct YAML; edit a `.md` file externally → Pull → verify SwiftData updated
- **Pro Gating**: Sign out of StoreKit sandbox → tap AI Assist button → paywall appears; complete purchase → feature becomes accessible; Restore Purchases → entitlement restored